

JNX-OS Backend Protection Prompt

CRITICAL: Read this BEFORE making ANY changes to the codebase.

This document serves as a mandatory checklist and reference guide to prevent breaking stable, production-ready functionality.

Pre-Development Checklist

Before making ANY changes, answer these questions:

1. Authentication & Authorization

- ☐ Will this change affect Clerk integration?
- ☐ Will this modify user/org synchronization logic?
- ☐ Does this touch webhook handlers or RBAC logic?
- ☐ Will this change authentication flows (login/signup/callback)?

If YES to any: Review Section A below.

2. Database Operations

- ☐ Will this modify database schema?
- ☐ Does this add new queries or transactions?
- ☐ Will this affect idempotent operations (UPSERT)?
- ☐ Does this touch multi-tenant data separation?

If YES to any: Review Section B below.

3. Security & Compliance

- ☐ Will this handle sensitive user data?
- ☐ Does this modify rate limiting or security headers?
- ☐ Will this affect GDPR compliance (deletion/export)?
- ☐ Does this log potentially sensitive information?

If YES to any: Review Section C below.

4. API Routes & Endpoints

- ☐ Will this create or modify API routes?
- ☐ Does this change error handling patterns?
- ☐ Will this affect webhook processing?
- ☐ Does this modify middleware logic?

If YES to any: Review Section D below.

5. UI/UX & Client Components

- ☐ Will this modify the JNX Dark design system?
- ☐ Does this change client-side authentication state?
- ☐ Will this affect the dashboard or admin panel?

- [] Does this modify loading/error states?

If **YES to any**: Review Section E below.

Section A: Authentication & Authorization

Protected System: Clerk Integration

Critical Files (DO NOT MODIFY without explicit reason):

```
lib/auth/clerk-client.ts      # Client-side Clerk utilities
lib/auth/clerk-server.ts     # Server-side Clerk utilities
lib/auth/helpers.ts          # requireAuth, requireAdmin
lib/auth/rbac.ts              # Role-based access control
middleware.ts                 # Route protection & redirects
```

How It Works:

1. **Clerk manages ALL authentication** (login, signup, OAuth, sessions)
2. **Webhooks sync data** from Clerk → Supabase (`/api/webhooks/clerk`)
3. **Middleware protects routes** based on auth status and role
4. **Server-side fallback** ensures dashboard access even during webhook delays

Rules:

-  **DO:** Use `requireAuth()` and `requireAdmin()` in server components
-  **DO:** Use `useIsAuthenticated()` and `useIsAdmin()` in client components
-  **DO:** Add new roles via Clerk dashboard (publicMetadata)
-  **DON'T:** Create custom authentication logic
-  **DON'T:** Bypass middleware for protected routes
-  **DON'T:** Store passwords or session tokens manually

Testing Requirements:

Before deploying auth changes:

1. Test signup flow (new user creation)
2. Test login flow (existing user)
3. Test admin access (role verification)
4. Check Clerk webhook logs for successful sync
5. Verify middleware redirects work correctly

Section B: Database Operations

Protected System: Supabase PostgreSQL (Schema v2)

Critical Files (DO NOT MODIFY without migration):

```
lib/db/schema-v2.sql          # Target database schema
lib/db/helpers.ts             # Idempotent UPSERT functions
MIGRATION_SIMPLE.sql          # GDPR migration script
CRITICAL_SCHEMA_RESTORE.md    # Schema verification guide
```

Core Schema (Production):

```
-- Users Table (Multi-Tenant)
users (
  id UUID PRIMARY KEY,
  clerk_user_id TEXT UNIQUE NOT NULL, -- Clerk sync
  org_id UUID REFERENCES orgs(id),
  email TEXT NOT NULL,
  full_name TEXT,
  role TEXT DEFAULT 'member',
  deleted_at TIMESTAMPTZ, -- GDPR soft-delete
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
)

-- Organizations Table
orgs (
  id UUID PRIMARY KEY,
  clerk_org_id TEXT UNIQUE,
  name TEXT NOT NULL,
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  created_at TIMESTAMPTZ DEFAULT NOW()
)
```

Idempotent Functions (MUST USE):

```
// From lib/db/helpers.ts
upsertUser(userData) // Atomic user create/update
upsertOrg(orgData) // Atomic org create/update
createUserWithOrg(userData) // Transactional user+org creation
syncUserFromClerk(clerkUserId) // Dashboard fallback sync
```

Rules:

- **✓ DO:** Use `upsertUser` / `upsertOrg` for all Clerk sync operations
- **✓ DO:** Use transactions for multi-table operations
- **✓ DO:** Test migrations on staging database first
- **✓ DO:** Use `deleted_at` for soft-deletes (GDPR)
- **✗ DON'T:** Use direct `INSERT` for user/org creation (race conditions!)
- **✗ DON'T:** Delete rows with `DELETE` (use soft-delete)
- **✗ DON'T:** Modify schema without creating migration file
- **✗ DON'T:** Remove `clerk_user_id` or `clerk_org_id` columns

Schema Change Process:

1. Create migration SQL file: `MIGRATION_[NAME].sql`
 2. Test on local Supabase instance
 3. Document changes in `CRITICAL_SCHEMA_RESTORE.md`
 4. Apply to production Supabase (SQL Editor)
 5. Update `lib/db/helpers.ts` if needed
 6. Run full test suite
-

Section C: Security & GDPR Compliance

Protected System: Security & Privacy

Critical Files (MODIFY WITH EXTREME CAUTION):

```
lib/security/rate-limit.ts      # Rate limiting middleware
lib/security/headers.ts        # Security headers (CSP, HSTS)
lib/privacy/redaction.ts        # PII redaction for logs
lib/privacy/deletion.ts         # GDPR Right to Erasure
lib/privacy/export.ts           # GDPR Data Portability
lib/observability/logger.ts     # Structured logging with redaction
```

Active Security Measures:

1. Rate Limiting:

- General: 100 req/15min per IP
- Auth: 10 req/15min per IP
- Strict: 20 req/15min per IP
- Implementation: In-memory (TODO: Redis for production scale)

2. Security Headers:

- Content Security Policy (CSP)
- HTTP Strict Transport Security (HSTS)
- X-Frame-Options, X-Content-Type-Options
- Referrer Policy

3. PII Protection:

- Email redaction: `user@example.com` → `u***@e***.com`
- Phone redaction: `+1234567890` → `+1*****7890`
- Automatic redaction in logs and error messages

4. GDPR Features:

- Soft-delete with `deleted_at` timestamp
- Data export in JSON format (`exportUserData`)
- Hard-delete option (`hardDeleteUser`)
- Audit logs for all deletion actions

Rules:

-  **DO:** Use `Logger` class for all logging (includes PII redaction)
-  **DO:** Apply rate limiting to all public API routes
-  **DO:** Use `applySecurityHeaders()` in API responses
-  **DO:** Log GDPR actions (deletion, export) to `audit_logs`
-  **DON'T:** Log raw user input without redaction
-  **DON'T:** Store credit card numbers or SSNs
-  **DON'T:** Hard-delete users without explicit user request
-  **DON'T:** Skip rate limiting on “internal” endpoints

GDPR Compliance Checklist:

For any feature handling user data:

- [] Can users export their data? (Right to Access)
- [] Can users delete their data? (Right to Erasure)

- [] Is data collection minimized? (Data Minimization)
- [] Are audit logs created? (Accountability)
- [] Is PII redacted in logs? (Privacy by Design)

Section D: API Routes & Webhooks

Protected System: Webhook & API Architecture

Critical Webhook Handler:

```
app/api/webhooks/clerk/route.ts # Clerk → Supabase sync
```

How Webhooks Work (CRITICAL - DO NOT BREAK):

1. **Clerk sends event** (user.created, org.updated, etc.)
2. **Svix verifies signature** (prevents tampering)
3. **Idempotent handler processes event:**
 - user.created → createUserWithOrg() (transactional)
 - user.updated → upsertUser() (idempotent)
 - organization.created/updated → upsertOrg() (idempotent)
4. **Errors trigger Clerk retry** (automatic recovery)
5. **Logs to system_events** for observability

Webhook Rules (NON-NEGOTIABLE):

-  **DO:** Use idempotent operations (UPSERT, not INSERT)
-  **DO:** Verify webhook signatures with Svix
-  **DO:** Return 200 OK even on handled errors (prevent retry loops)
-  **DO:** Log ALL webhook events to system_events
-  **DO:** Use transactions for multi-step operations
-  **DON'T:** Use direct INSERT (causes race conditions)
-  **DON'T:** Skip signature verification (security risk)
-  **DON'T:** Throw errors without logging context
-  **DON'T:** Modify webhook event types Clerk sends

API Route Best Practices:

```
// Template for new API routes
import { NextRequest, NextResponse } from 'next/server';
import { currentUser } from '@clerk/nextjs/server';
import { Logger } from '@lib/observability/logger';
import { applyRateLimit } from '@lib/security/rate-limit';

const logger = new Logger('api/your-route');

export async function POST(req: NextRequest) {
  try {
    // 1. Rate limiting
    const identifier = req.ip || 'anonymous';
    const rateLimitResult = await applyRateLimit('general', identifier);
    if (!rateLimitResult.success) {
      return NextResponse.json(
        { error: 'Too many requests' },
        { status: 429 }
      );
    }

    // 2. Authentication (if required)
    const user = await currentUser();
    if (!user) {
      return NextResponse.json(
        { error: 'Unauthorized' },
        { status: 401 }
      );
    }

    // 3. Input validation
    const body = await req.json();
    // ... validate with Zod or similar

    // 4. Business logic
    // ... your code here

    // 5. Logging
    logger.info('Operation successful', { userId: user.id });

    return NextResponse.json({ success: true });
  } catch (error) {
    logger.error('Operation failed', { error });
    return NextResponse.json(
      { error: 'Internal server error' },
      { status: 500 }
    );
  }
}
```

Section E: UI/UX & Client Components

Protected System: JNX Dark Design System

Core Design Tokens:

```
/* From tailwind.config.ts */
Colors:
jnx-dark: #0a0f1e      // Background
jnx-darker: #060a14    // Deeper background
cyan-500: #06b6d4      // Primary accent
blue-600: #2563eb      // Secondary accent
slate-800: #1e293b     // Cards/containers
```

Core UI Components (DO NOT BREAK):

```
components/ui/button-primary.tsx    # Gradient CTA button
components/ui/button-secondary.tsx  # Subtle action button
components/ui/input-field.tsx       # Styled form input
components/ui/feature-card.tsx      # Feature highlight card
components/ui/terminal-box.tsx      # Code/status display
components/ui/status-badge.tsx      # Connection status indicator
```

Dashboard Architecture:

```
app/app/page.tsx          # Server-side entry (requireAuth)
app/app/dashboard-client.tsx # Client-side UI
app/app/dashboard-setup.tsx # Loading state with retry logic
```

Client-Side Auth Pattern (MUST FOLLOW):

```
'use client';

import { useUser, useClerk } from '@clerk/nextjs';

export function MyComponent() {
  const { user, isLoading } = useUser();
  const { signOut } = useClerk();

  // 1. Show loading state
  if (!isLoading) {
    return <div>Loading...</div>;
  }

  // 2. Handle unauthenticated (should not happen due to middleware)
  if (!user) {
    return <div>Please log in</div>;
  }

  // 3. Render authenticated UI
  return (
    <div>
      <p>Welcome, {user.firstName}</p>
      <button onClick={() => signOut()}>Sign Out</button>
    </div>
  );
}
```

Rules:

-  **DO:** Use existing JNX UI components for consistency
-  **DO:** Follow dark theme color palette
-  **DO:** Show loading states during data fetching
-  **DO:** Handle error states with clear user messaging
-  **DO:** Use `useUser()` for client-side auth state
-  **DON'T:** Create custom button/input styles (use design system)
-  **DON'T:** Use light theme colors (breaks design consistency)
-  **DON'T:** Skip loading states (causes hydration errors)
-  **DON'T:** Access `user.emailAddresses` (use `user.email` directly)



Absolutely Forbidden Actions

These will **BREAK** production. **DO NOT DO THIS:**

1. Database:

-  Remove `clerk_user_id` or `clerk_org_id` columns
-  Change primary key structures (UUID → other)
-  Drop foreign key constraints
-  Remove `deleted_at` column (breaks GDPR)

2. Authentication:

-  Modify webhook signature verification
-  Skip `requireAuth()` on protected routes
-  Create custom session management
-  Store Clerk secret keys in client-side code

3. Security:

-  Disable rate limiting “temporarily”
-  Log unredacted PII
-  Remove security headers
-  Skip input validation on API routes

4. Code Quality:

-  Use `any` type in TypeScript (use proper types)
-  Suppress TypeScript errors with `@ts-ignore`
-  Remove error handling to “simplify” code
-  Hard-code environment variables

✓ Safe Development Patterns

How to Add New Features WITHOUT Breaking Things:

1. New Authentication Route:

```
// app/api/protected-feature/route.ts
import { requireAuth } from '@lib/auth/helpers';

export async function GET() {
  const { user, jnxUser } = await requireAuth();
  // Your logic here - user is guaranteed to exist
  return NextResponse.json({ data: 'something' });
}
```

2. New Database Query:

```
// lib/db/your-feature.ts
import { createSupabaseServerClient } from '@lib/supabase/server';

export async function getYourData(userId: string) {
  const supabase = createSupabaseServerClient();

  const { data, error } = await supabase
    .from('your_table')
    .select('*')
    .eq('user_id', userId)
    .is('deleted_at', null); // Respect soft-deletes

  if (error) throw error;
  return data;
}
```

3. New UI Component:

```
// components/ui/your-component.tsx
import { cn } from '@lib/utils';

interface YourComponentProps {
  title: string;
  className?: string;
}

export function YourComponent({ title, className }: YourComponentProps) {
  return (
    <div className={cn(
      'bg-slate-900/40 border border-slate-800/60 rounded-xl p-6',
      className
    )}>
      <h3 className="text-xl font-semibold text-white">{title}</h3>
    </div>
  );
}
```

4. Extend RBAC:

```
// lib/auth/rbac.ts (add new permission)
export const PERMISSIONS = {
  admin: ['manage_users', 'view_analytics', 'your_new_permission'],
  member: ['view_dashboard'],
} as const;

// Use in component:
import { useIsAdmin } from '@lib/auth/clerk-client';

if (useIsAdmin()) {
  // Show admin-only UI
}
```

Testing Requirements

Before ANY deployment, verify:

1. Build & Type Safety:

```
cd nextjs_space
yarn build # Must succeed with 0 errors
```

2. Authentication Flow:

- [] New user signup works
- [] Existing user login works
- [] Dashboard loads within 3 seconds
- [] Admin panel accessible to admin role only
- [] Webhooks sync user data to Supabase

3. Database Operations:

- [] No duplicate key errors in logs
- [] Transactions commit successfully
- [] Soft-deletes preserve referential integrity
- [] Audit logs created for sensitive operations

4. Security:

- [] Rate limiting triggers after threshold
- [] Unauthorized requests return 401
- [] PII is redacted in console logs
- [] Webhook signatures verified

5. UI/UX:

- [] Loading states display correctly
- [] Error messages are user-friendly
- [] Dark theme consistent across pages
- [] No hydration errors in console

Reference Documentation

Read **BEFORE** modifying related systems:

- `README.md` - Project overview & quick start
 - `ARCHITECTURE.md` - System design & data flow
 - `BACKEND_CONTRACT.md` - Non-negotiable backend rules
 - `CLERK_SETUP.md` - Clerk configuration guide
 - `GDPR_COMPLIANCE.md` - Privacy & compliance requirements
 - `CRITICAL_SCHEMA_RESTORE.md` - Database schema verification
 - `DELIVERY_SUMMARY.md` - Enterprise rebuild report
-

Success Criteria

Your change is production-ready when:

- ☒ All builds pass (`yarn build` succeeds)
 - ☒ No TypeScript errors (`tsc --noEmit`)
 - ☒ Tests pass (when implemented)
 - ☒ Authentication flows work
 - ☒ No 500 errors in server logs
 - ☒ Webhook success rate = 100%
 - ☒ Dashboard loads < 3 seconds
 - ☒ No console errors on client
 - ☒ Code review approved
 - ☒ This checklist completed
-

When in Doubt

If you're unsure about a change:

1. **Check this document first** - Does it violate any rules?
2. **Read the referenced docs** - Is there existing guidance?
3. **Test locally** - Does it work in development?
4. **Review the code** - Are you following existing patterns?
5. **Ask for review** - Get a second opinion before deploying

Remember: It's better to ask than to break production.

Change Log Template

Use this when documenting changes:

Change: [Brief description]****Date:**** YYYY-MM-DD****Author:**** [Your name]****Files Modified:****

- path/to/file1.ts
- path/to/file2.tsx

****What Changed:****

- Added/Modified/Removed X
- Updated Y to support Z

****Why:****

- [Business justification]

****Testing:****

- ☐ Local build passes
- ☐ Feature tested manually
- ☐ No regressions observed

****Risks:****

- [Any potential issues]

****Rollback Plan:****

- [How to revert if needed]

Last Updated: 2024-12-27**Version:** 2.0 (Post-Enterprise Rebuild)**Status:** ☒ Production Ready

Final Note

This system is **production-ready** and **battle-tested**. The architecture is stable, secure, and GDPR-compliant.

Your job: Build amazing features on top of this foundation without breaking what already works.

Good luck, and happy coding! 🚀